Human Verification: `Containers.CoKleisli`

CoKleisli Category and Canonical Strength

Machine-generated from `lean/CoKleisli.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.CoKleisli` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **1**

2 CoKleisli extension **2**
- 2.1 Definition: `extend` . . . 2
- 2.2 Theorem: `extend_eq_map_comult` . . . 2
- 2.3 Theorem: `extend_counit` . . . 2
- 2.4 Theorem: `counit_extend` . . . 3
- 2.5 Theorem: `extend_extend` . . . 3

3 CoKleisli composition **3**
- 3.1 Definition: `cokleisli_comp` . . . 3
- 3.2 Theorem: `cokleisli_comp_counit` . . . 4
- 3.3 Theorem: `cokleisli_counit_comp` . . . 4
- 3.4 Theorem: `cokleisli_assoc` . . . 4

4 Canonical strength **5**
- 4.1 Definition: `strength` . . . 5
- 4.2 Theorem: `strength_natural_left` . . . 5
- 4.3 Theorem: `strength_natural_right` . . . 5
- 4.4 Theorem: `strength_counit` . . . 6

5 Translation conventions **6**

1 Overview

This file develops the coKleisli structure and canonical strength for directed container comonads, extending `Containers.Directed`.

A **directed container** $D = (S, P, o, \downarrow, \oplus)$ gives rise to a comonad on its extension $\mathbf{Ext}\, D$ (a comonad on **Type**). The coKleisli category of this comonad has as morphisms $X \to Y$ the functions $\mathbf{Ext}\, D\, X \to Y$, composed via the *extension* operation. This file defines `extend`,

`cokleisli_comp`, and the canonical strength $\sigma : W(A \times B) \to A \times W(B)$, and proves the categorical laws.

Everything is **Type**-level, Lean 4 core, no Mathlib.

2 CoKleisli extension

2.1 Definition: `extend`

Definition 2.1 (CoKleisli extension)**.** Given $f : \mathbf{Ext}\, D\, X \to Y$, the **coKleisli extension** $\texttt{extend}\, f : \mathbf{Ext}\, D\, X \to \mathbf{Ext}\, D\, Y$ applies f at every position via comultiplication:

$$\texttt{extend}\, f\, \langle s, v\rangle \;=\; \langle s,\ \lambda p.\ f\langle s\downarrow p,\ \lambda q.\ v(p \oplus q)\rangle\rangle.$$

Equivalently, $\texttt{extend}\, f = \mathbf{Ext}\, f \circ \delta$, where δ is the comonad comultiplication.

Lean.

```
def extend {X Y : Type} (f : Ext D.toContainer X → Y) :
    Ext D.toContainer X → Ext D.toContainer Y :=
  fun p ⇒ ⟨p.1, fun q ⇒ f ⟨D.sub p.1 q, fun r ⇒ p.2 (D.shift p.1 q
    r)⟩⟩
```

Haskell.

```
-- In the coKleisli category of a comonad W, extend lifts
-- f :: W x → y  to  extend f :: W x → W y.
-- For a directed container this is:
extend :: DirectedContainer s p → (Ext s p x → y) → Ext s p x →
   Ext s p y
extend d f (Ext s v) = Ext s (\q → f (Ext (sub d s q) (\r → v (shift
    d s q r))))
```

2.2 Theorem: `extend_eq_map_comult`

Proposition 2.2 (Extension as map after comult)**.** For any $f : \mathbf{Ext}\, D\, X \to Y$,

$$\texttt{extend}\, f \;=\; \mathbf{Ext}.\texttt{map}\, f \circ \delta,$$

where $\delta = \texttt{comult}$ is the comonad comultiplication. This holds definitionally (by `rfl`).

Lean.

```
theorem extend_eq_map_comult {X Y : Type} (f : Ext D.toContainer X →
   Y) :
    D.extend f = Ext.map f  ∘  D.comult
```

Haskell.

```
-- extend f = fmap f . comult
-- This is the standard relationship between extend and (>>=) in a
   comonad.
```

2.3 Theorem: `extend_counit`

Theorem 2.3 (`extend` of the counit is the identity)**.**

$$\texttt{extend}\, \varepsilon \;=\; \mathrm{id}.$$

This is the *right unit law* of the coKleisli category.

Lean.

```
theorem extend_counit {X : Type} :
    D.extend (D.counit (X := X)) = id
```

Haskell.

```
-- Comonad law (right unit):
-- extend extract = id
```

2.4 Theorem: `counit_extend`

Theorem 2.4 (Counit is a left inverse of extension)**.** For every $f : \mathbf{Ext}\, D\, X \to Y$,

$$\varepsilon \circ \mathtt{extend}\, f \;=\; f.$$

This characterises $\mathtt{extend}\, f$ as the unique map h satisfying $\varepsilon \circ h = f$ (the universal property of extension).

Lean.

```
theorem counit_extend {X Y : Type} (f : Ext D.toContainer X → Y) :
    D.counit ∘ D.extend f = f
```

Haskell.

```
-- Comonad law (left unit):
-- extract . extend f = f
```

2.5 Theorem: `extend_extend`

Theorem 2.5 (Extension respects coKleisli composition)**.** For $f : \mathbf{Ext}\, D\, X \to Y$ and $g : \mathbf{Ext}\, D\, Y \to Z$,

$$\mathtt{extend}(g \circ \mathtt{extend}\, f) \;=\; \mathtt{extend}\, g \circ \mathtt{extend}\, f.$$

This is the *associativity law* of the coKleisli category; it follows from coassociativity of the comonad.

Lean.

```
theorem extend_extend {X Y Z : Type}
    (f : Ext D.toContainer X → Y)
    (g : Ext D.toContainer Y → Z) :
    D.extend (g ∘ D.extend f) = D.extend g ∘ D.extend f
```

Haskell.

```
-- Comonad law (associativity / fusion):
-- extend (g . extend f) = extend g . extend f
```

3 CoKleisli composition

3.1 Definition: `cokleisli_comp`

Definition 3.1 (CoKleisli composition)**.** Given coKleisli morphisms $f : \mathbf{Ext}\, D\, X \to Y$ and $g : \mathbf{Ext}\, D\, Y \to Z$, their **coKleisli composite** is

$$g \mathbin{.\!*} f \;:=\; g \circ \mathtt{extend}\, f \;:\; \mathbf{Ext}\, D\, X \to Z.$$

Lean.

```
def cokleisli_comp {X Y Z : Type}
    (f : Ext D.toContainer X → Y)
    (g : Ext D.toContainer Y → Z) :
    Ext D.toContainer X → Z :=
  g ∘ D.extend f
```

Haskell.

```haskell
-- CoKleisli composition  (=>>) in Control.Comonad:
-- (=>>) :: (w a -> b) -> (w b -> c) -> w a -> c
cokleisliComp :: (Ext s p x -> y) -> (Ext s p y -> z) -> Ext s p x ->
   z
cokleisliComp f g = g . extend f
```

3.2 Theorem: `cokleisli_comp_counit`

Theorem 3.2 (Right unit of CoKleisli composition)**.** For every $f : \mathbf{Ext}\, D\, X \to Y$,

$$f \mathbin{.*} \varepsilon \;=\; f.$$

The counit ε is a right identity for coKleisli composition.

Lean.

```lean
theorem cokleisli_comp_counit {X Y : Type}
    (f : Ext D.toContainer X → Y) :
    D.cokleisli_comp (D.counit) f = f
```

Haskell.

```haskell
-- Right unit:  f =>> extract = f
```

3.3 Theorem: `cokleisli_counit_comp`

Theorem 3.3 (Left unit of CoKleisli composition)**.** For every $f : \mathbf{Ext}\, D\, X \to Y$,

$$\varepsilon \mathbin{.*} f \;=\; f.$$

The counit ε is a left identity for coKleisli composition.

Lean.

```lean
theorem cokleisli_counit_comp {X Y : Type}
    (f : Ext D.toContainer X → Y) :
    D.cokleisli_comp f D.counit = f
```

Haskell.

```haskell
-- Left unit:  extract =>> f = f
```

3.4 Theorem: `cokleisli_assoc`

Theorem 3.4 (Associativity of CoKleisli composition)**.** For $f : \mathbf{Ext}\, D\, X \to Y$, $g : \mathbf{Ext}\, D\, Y \to Z$, $h : \mathbf{Ext}\, D\, Z \to W$,

$$(h \mathbin{.*} g) \mathbin{.*} f \;=\; h \mathbin{.*} (g \mathbin{.*} f).$$

Lean.

```lean
theorem cokleisli_assoc {X Y Z W : Type}
    (f : Ext D.toContainer X → Y)
    (g : Ext D.toContainer Y → Z)
    (h : Ext D.toContainer Z → W) :
    D.cokleisli_comp (D.cokleisli_comp f g) h
      = D.cokleisli_comp f (D.cokleisli_comp g h)
```

Haskell.

```haskell
-- Associativity:  (f =>> g) =>> h = f =>> (g =>> h)
```

4 Canonical strength

4.1 Definition: `strength`

Definition 4.1 (Canonical strength)**.** The **canonical strength** of the directed container comonad is a natural transformation

$$\sigma_{A,B} : \mathbf{Ext}\, D\, (A \times B) \longrightarrow A \times \mathbf{Ext}\, D\, B,$$

defined by

$$\sigma_{A,B}\langle s, v\rangle = (\pi_1(v(o(s))),\ \langle s, \pi_2 \circ v\rangle).$$

The first component extracts the A-value at the root; the second keeps the shape and strips each labelled pair to its B-component.

Lean.

```
def strength (A B : Type) :
    Ext D.toContainer (A ⊗ B) → A ⊗ Ext D.toContainer B :=
  fun p ⇒ ⟨(p.2 (D.root p.1)).1, ⟨p.1, fun q ⇒ (p.2 q).2⟩⟩
```

Haskell.

```
-- Canonical comonad strength:  sigma :: w (a, b) → (a, w b)
-- For a directed container:
strength :: DirectedContainer s p → Ext s p (a, b) → (a, Ext s p b)
strength d (Ext s v) = (fst (v (root d s)), Ext s (snd . v))
```

Fidelity note. Every comonad on a cartesian category is canonically strong. Here the A-component is extracted at the *root* position $o(s)$, which is well-typed because the counit extracts at the root. The Lean source has a minor syntactic issue in line 166 (a missing opening angle bracket); the intent is clear from context and from the surrounding docstring.

4.2 Theorem: `strength_natural_left`

Theorem 4.2 (Strength naturality in A)**.** For every $f : A \to A'$,

$$(f \times \mathrm{id}) \circ \sigma_{A,B} = \sigma_{A',B} \circ \mathbf{Ext}\, D\, (f \times \mathrm{id}).$$

Lean.

```
theorem strength_natural_left {A A' B : Type} (f : A → A') :
    (fun p : A ⊗ Ext D.toContainer B ⇒ (f p.1, p.2))  ∘  D.strength A
        B
      = D.strength A' B  ∘  Ext.map (fun ab : A ⊗ B ⇒ (f ab.1, ab.2))
```

Haskell.

```
-- Left naturality:
-- first f . strength = strength . fmap (first f)
-- where first f (a, b) = (f a, b)
```

4.3 Theorem: `strength_natural_right`

Theorem 4.3 (Strength naturality in B)**.** For every $g : B \to B'$,

$$(\mathrm{id} \times \mathbf{Ext}\, D\, g) \circ \sigma_{A,B} = \sigma_{A,B'} \circ \mathbf{Ext}\, D\, (\mathrm{id} \times g).$$

Lean.

```
theorem strength_natural_right {A B B' : Type} (g : B → B') :
    (fun p : A ⊗ Ext D.toContainer B ⇒ (p.1, Ext.map g p.2))  ∘  D.
       strength A B
      = D.strength A B'  ∘  Ext.map (fun ab : A ⊗ B ⇒ (ab.1, g ab.2))
```

Haskell.

```haskell
-- Right naturality:
-- second (fmap g) . strength = strength . fmap (second g)
-- where second g (a, b) = (a, g b)
```

4.4 Theorem: `strength_counit`

Theorem 4.4 (Strength–counit coherence)**.**

$$(\mathrm{id}_A \times \varepsilon_B) \circ \sigma_{A,B} \;=\; \varepsilon_{A \times B}.$$

Extracting from the B-component of the strength gives the same result as extracting directly from the $A \times B$-tree.

Lean.

```
theorem strength_counit {A B : Type} :
    (fun p : A ⊗ Ext D.toContainer B ⇒ (p.1, D.counit p.2))  ∘  D.
       strength A B
      = D.counit (X := A ⊗ B)
```

Haskell.

```haskell
-- Strength-counit coherence:
-- second extract . strength = extract
-- i.e.  (id *** extract) . sigma = extract
```

5 Translation conventions

The following conventions apply throughout this document.

1. **Directed container notation.** A directed container $D = (S, P, o, \downarrow, \oplus)$ has shapes S, positions P, root $o : S \to P$, sub-shape $(\downarrow) : S \to P(s) \to S$, and shift $(\oplus) : (s : S) \to P(s) \to P(s \downarrow p) \to P(s)$. The laws D1–D5 from `Containers.Directed` are used implicitly in the proofs.

2. **CoKleisli = Kleisli of the opposite monad.** The coKleisli category of a comonad W has objects the types and morphisms $X \to Y$ the functions $W(X) \to Y$; identity is ε and composition is `cokleisli_comp`.

3. **Haskell comonad vocabulary.** Haskell's `Control.Comonad` uses `extract` for ε and `extend` for the coKleisli lift; the operator `=>>` is coKleisli composition in the left-to-right order.

4. **Proof bodies omitted.** Only definitions and theorem *statements* are translated. Lean checks the proofs; the human verifier checks that the *statements* match the intended mathematics.